

R Lab8. Análisis estadístico de datos RNA-Seq con R

Bioinformática. Ingeniería Biomédica

En este laboratorio vamos a introducir herramientas útiles para analizar datos de conteo procedentes de la metodología de secuenciación masiva RNA-Seq. Vamos a utilizar el paquete `edgeR` desarrollado específicamente en este contexto y disponible en Bioconductor. Instala y carga esta librería.

Solución:

```
if(!("edgeR" %in% installed.packages()))  
  BiocManager::install("edgeR")  
library("edgeR")
```

Lectura de datos

Vamos a utilizar como ejemplo los datos de conteo del paquete de datos `pasilla` disponible en Bioconductor. Este conjunto de datos procede de un experimento del organismo procariota *Drosophila melanogaster*, en el que se evaluaron los genes ortólogos que codifican proteínas de unión a partir de RNA interferente: molécula de RNA que suprime la expresión de genes específicos mediante mecanismos conocidos globalmente como ribointerferencia o interferencia por RNA (siRNA, miRNA, piRNA). Estos datos también están disponibles en GEO, con identificador *GSE18508* (<https://www.ncbi.nlm.nih.gov/geo/query/acc.cgi?acc=gse18508>).

1. Los datos de conteo están en el fichero "pasilla_gene_counts.tsv" disponibles en el campus. Lee el fichero utilizando la función `read.table()`. Puedes utilizar el argumento `row.names=1` para indicar que la primera columna contiene el nombre de las filas (genes). ¿Cuántos genes se han evaluado en esta matriz?

Solución:

```
rawCountTable <- read.table("pasilla_gene_counts.tsv", header = TRUE, row.names = 1)  
dim(rawCountTable)
```

```
## [1] 14599 7
```

```
head(rawCountTable)
```

```
##      untreated1 untreated2 untreated3 untreated4 treated1 treated2  
## FBgn0000003      0      0      0      0      0      0  
## FBgn0000008     92     161      76      70     140     88  
## FBgn0000014      5      1      0      0      4      0  
## FBgn0000015      0      2      1      2      1      0  
## FBgn0000017    4664    8714    3564    3150    6205    3072  
## FBgn0000018     583     761     245     310     722     299  
##      treated3  
## FBgn0000003      1  
## FBgn0000008     70  
## FBgn0000014      0  
## FBgn0000015      0  
## FBgn0000017    3334  
## FBgn0000018     308
```

Notad que los datos son enteros, ya que resultan de un conteo. La matriz de conteos tiene unas dimensiones de 14599 filas (genes) por 7 columnas (muestras).

1.1. En este mismo paquete tenemos disponibles también los datos fenotípicos correspondientes a las condiciones experimentales y de secuenciación. Se encuentran en el fichero "pasilla_sample_annotation.csv", que tenéis disponible en el campus y podéis leer utilizando la función `read.csv()`,

```
SampleAnno <- read.csv("pasilla_sample_annotation.csv")  
SampleAnno
```

```
##      file condition      type number.of.lanes total.number.of.reads
## 1  treated1fb   treated single-read          5      35158667
## 2  treated2fb   treated paired-end           2      12242535 (x2)
## 3  treated3fb   treated paired-end           2      12443664 (x2)
## 4  untreated1fb untreated single-read          2       17812866
## 5  untreated2fb untreated single-read          6       34284521
## 6  untreated3fb untreated paired-end          2       10542625 (x2)
## 7  untreated4fb untreated paired-end          2       12214974 (x2)
## exon.counts
## 1  15679615
## 2  15620018
## 3  12733865
## 4  14924838
## 5  20764558
## 6  10283129
## 7  11653031
```

La primera columna corresponde a las muestras. La 2ª y 3ª corresponden a la condición de cada muestra (tratado o no tratado) y al tipo de librería de secuenciación (*paired-end* o *single-end*). El resto de columnas corresponden al número total de lecturas de cada muestra, al conteo de exones y al número de 'carriles', información relativa al proceso de secuenciación.

Filtrado de los datos

Antes de hacer un análisis de expresión diferencial con datos RNA-Seq es frecuente eliminar los genes que presentan niveles de expresión muy bajos en todos los arrays considerados, es decir hacer un filtrado eliminando los genes irrelevantes.

Hay muchas formas de abordar estos filtros: eliminar genes cuyo nivel de expresión está por debajo de un umbral fijado, genes con 0 conteo en todas las muestras, etc.

1. Dado que algunos genes no se han detectado en ninguna de las muestras, toda la fila correspondiente puede ser 0. Elimina estos genes del conjunto de datos a analizar. ¿Con cuántos genes te quedas?

Solución: Una solución posible sería, utilizando la función `rowSums()` para detectar las filas con algún valor distinto de 0 en la matriz de conteo.

```
keep <- rowSums(rowCountTable) > 0
filtCount <- rowCountTable[keep, ]
dim(filtCount)
```

```
## [1] 12359    7
```

Nos quedamos con 12359 genes.

2. Representa las distribuciones de las muestras (en escala log2), en este nuevo conjunto de genes.

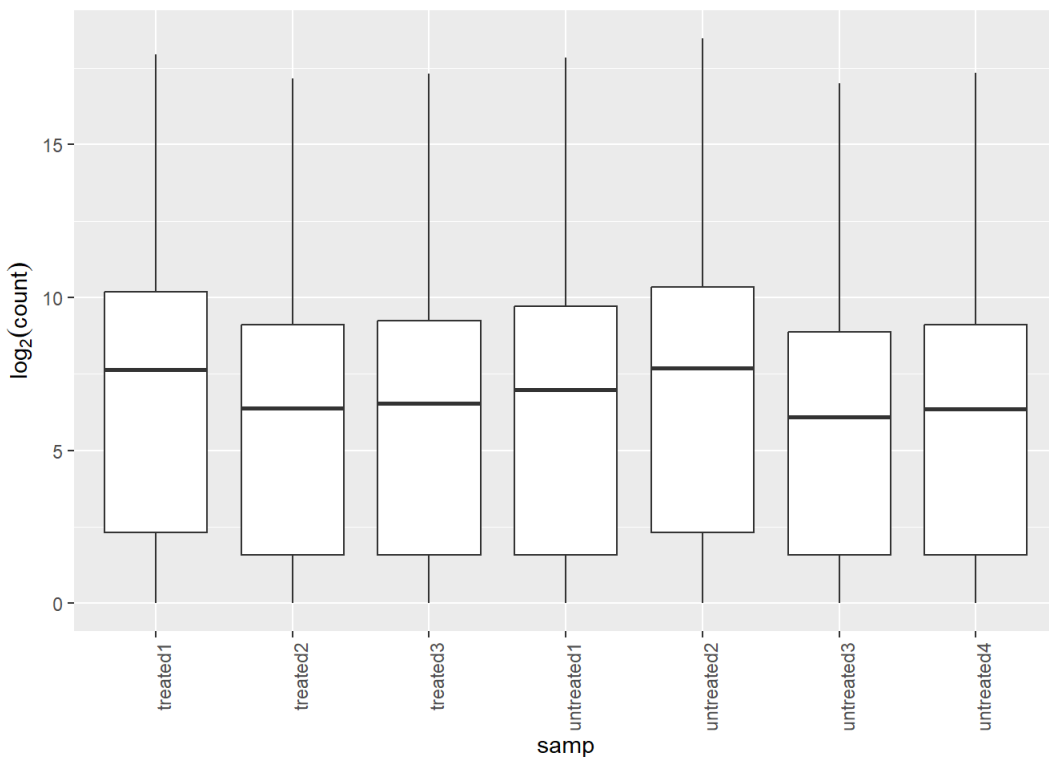
Solución:

Utilizando `ggplot2` y un diagrama de cajas múltiple,

```
library(ggplot2)

# Convertir filtCount a formato largo
df <- data.frame(
  samp = rep(colnames(filtCount), each = nrow(filtCount)),
  value = as.vector(log2(as.matrix(filtCount)+1)) # hay que transformar el data.frame a matrix
)

# Hacer el boxplot
ggplot(df, aes(x = samp, y = value)) +
  geom_boxplot() +
  theme(axis.text.x = element_text(angle = 90, hjust = 1)) +
  ylab(expression(log2(count)))
```



Normalización

Utilizamos el paquete `edgeR` para normalizar los datos utilizando distintos métodos y comparar sus resultados.

1. El primer paso es crear un objeto de la clase `DGEList` a partir de la matriz de conteos sin transformar. Utiliza la función `edgeR::DGEList()` que tiene como argumento la matriz de conteos sin transformar. Además, podéis añadir la información de los grupos con el argumento `group` en el que asignais un vector con el grupo al que pertenece cada muestra.

Solución:

```
condition<-as.factor(gsub("[0-9]", "", colnames(rawCountTable)))
dge2 <- DGEList(filtCount, group = as.numeric(condition))
dge2
```

```
## An object of class "DGEList"
## $counts
##      untreated1 untreated2 untreated3 untreated4 treated1 treated2
## FBgn0000003      0      0      0      0      0      0
## FBgn0000008     92     161      76      70     140     88
## FBgn0000014      5      1      0      0      4      0
## FBgn0000015      0      2      1      2      1      0
## FBgn0000017    4664     8714     3564     3150     6205     3072
##      treated3
## FBgn0000003      1
## FBgn0000008     70
## FBgn0000014      0
## FBgn0000015      0
## FBgn0000017    3334
## 12354 more rows ...
##
## $samples
##      group lib.size norm.factors
## untreated1  2 13972512      1
## untreated2  2 21911438      1
## untreated3  2 8358426      1
## untreated4  2 9841335      1
## treated1    1 18670279      1
## treated2    1 9571826      1
## treated3    1 10343856      1
```

Notad que el objeto `dge2` almacena la información en forma de lista con dos elementos: el primero es la matriz de conteos (`counts`), y el segundo es la matriz de muestras (`samples`). Esta última matriz la columna `norm.factors` almacena los valores de los factores de normalización para cada muestra. En este caso 1 puesto que no hemos aplicado ningún método aún.

2. Vamos a utilizar el método de normalización por la media recortada de los M-valores (TMM), que es uno de los más populares. Calculamos los valores del factor de normalización para cada muestra utilizando la función `calcNormFactors()` . Requiere dos argumentos, el objeto de la clase

DGEList creado en el apartado anterior, y el argumento `method`, en donde se especifica el método de normalización a utilizar, en este caso "TMM". ¿Qué tipo de objeto se obtiene?

Solución: Esta función cambia los valores de la variable `norm.factors` del elemento `samples` del objeto de la clase `DGEList`, por lo que el resultado de la función `calcNormFactors()` le voy a reasignar al objeto `dge2`.

```
dge2 <- calcNormFactors(dge2, method = "TMM")
dge2
```

```
## An object of class "DGEList"
## $counts
##      untreated1 untreated2 untreated3 untreated4 treated1 treated2
## FBgn0000003      0      0      0      0      0      0
## FBgn0000008     92     161      76      70     140     88
## FBgn0000014      5      1      0      0      4      0
## FBgn0000015      0      2      1      2      1      0
## FBgn0000017    4664     8714     3564     3150     6205     3072
##      treated3
## FBgn0000003      1
## FBgn0000008     70
## FBgn0000014      0
## FBgn0000015      0
## FBgn0000017    3334
## 12354 more rows ...
##
## $samples
##      group lib.size norm.factors
## untreated1  2 13972512  0.9995731
## untreated2  2 21911438  1.0081519
## untreated3  2 8358426   0.9843974
## untreated4  2 9841335   0.9525077
## treated1    1 18670279  1.0651817
## treated2    1 9571826   0.9957012
## treated3    1 10343856  0.9978557
```

Para aplicar otros métodos de normalización distintos, podéis modificar el argumento `method` cuyos valores pueden ser `c("TMM", "TMMwsp", "RLE", "upperquartile", "none")`.

3. Obtenemos la matriz de conteos normalizada utilizando la función `cpm()`. El resultado serán los datos crudos (sin transformación logaritmica) ya normalizados. Aplica la transformación $\log_2$ sobre estos datos.

Solución:

```
# matriz de conteo normalizada
pseudo.TMM <- log2(cpm(dge2) + 1)
head(pseudo.TMM)
```

```
##      untreated1 untreated2 untreated3 untreated4 treated1 treated2
## FBgn0000003  0.0000000 0.00000000  0.000000  0.0000000 0.0000000 0.000000
## FBgn0000008  2.9235616 3.05108431  3.355684  3.0819370 3.0071398 3.355205
## FBgn0000014  0.4414817 0.06387459  0.000000  0.0000000 0.2643971 0.000000
## FBgn0000015  0.0000000 0.12504066  0.165476  0.2790044 0.0707788 0.000000
## FBgn0000017  8.3877623 8.62744668  8.762064  8.3967664 8.2900608 8.336853
## FBgn0000018  5.4176028 5.14770909  4.943749  5.0904462 5.2212849 5.016691
##      treated3
## FBgn0000003  0.1334103
## FBgn0000008  2.9601121
## FBgn0000014  0.0000000
## FBgn0000015  0.0000000
## FBgn0000017  8.3398926
## FBgn0000018  4.9467363
```

4. Comparamos las distribuciones antes y después de la normalización.

4.1. Necesitamos un `data.frame` con tres variables: `samp` con la muestra, `value` para el conteo y `method` que contenga el método de normalización utilizado. Utilizamos la función `rbind()`, para concatenar por filas dos `data.frame`, uno con los datos sin normalizar y otro normalizado.

```
# data.frame para los datos sin normalizar
df.raw <- data.frame(
  samp = rep(colnames(filtCount), each = nrow(filtCount)),
  value = as.vector(log2(as.matrix(filtCount)+1)), # hay que transformar el data.frame a matrix
  method = "raw"
)

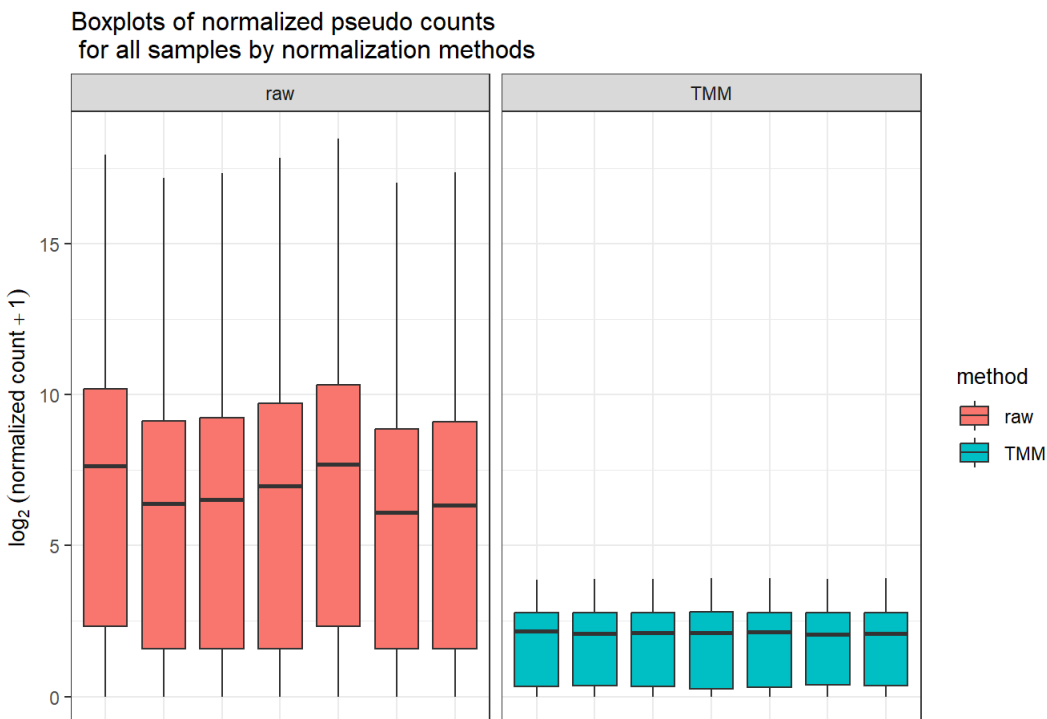
# data.frame para los datos normalizados
df.TMM <- data.frame(
  samp = rep(colnames(pseudo.TMM), each = nrow(pseudo.TMM)),
  value = as.vector(log2(as.matrix(pseudo.TMM)+1)), # hay que transformar el data.frame a matrix
  method = "TMM"
)

df <- rbind(df.raw,df.TMM)
head(df.TMM)
```

```
##      samp  value method
## 1 untreated1 0.000000   TMM
## 2 untreated1 1.9721638   TMM
## 3 untreated1 0.5275525   TMM
## 4 untreated1 0.000000   TMM
## 5 untreated1 3.2307813   TMM
## 6 untreated1 2.6820345   TMM
```

4.2. Utilizamos boxplots para comparar las distribuciones pre- y post-normalización.

```
p <- ggplot(data=df, aes(x=samp, y=value, fill=method)) +
  geom_boxplot() + theme_bw() +
  ggtitle("Boxplots of normalized pseudo counts\n for all samples by normalization methods") +
  facet_grid(. ~ method) +
  ylab(expression(log[2] ~ (normalized ~ count + 1))) + xlab("") +
  theme(title = element_text(size=10), axis.text.x = element_blank(),
        axis.ticks.x = element_blank())
print(p)
```



Análisis de expresión diferencial

Vamos a comparar los resultados de dos métodos diferentes para obtener genes diferencialmente expresados: un test exacto basado en el modelo binomial negativo y otro basado en el ajuste de un modelo lineal general por gen similar al método `limma` con microarrays y que nos permite tener en cuenta la réplica.

Test exacto para comparar dos grupos

1.1. Creamos un objeto de tipo `DGEList` utilizando la matriz de conteo original y en el que incorporamos la información sobre el tipo de muestra: `condition`. Utiliza el argumento `remove.zeros=TRUE` para eliminar los genes que no se han expresado en ninguna de las muestras.

Solución:

```
condition<-as.factor(gsub("[0-9]", "", colnames(rawCountTable)))
(dge <- DGEList(rawCountTable, group = condition, remove.zeros = TRUE))
```

```
## An object of class "DGEList"
## $counts
##      untreated1 untreated2 untreated3 untreated4 treated1 treated2
## FBgn0000003      0      0      0      0      0      0
## FBgn0000008     92     161      76      70     140     88
## FBgn0000014      5      1      0      0      4      0
## FBgn0000015      0      2      1      2      1      0
## FBgn0000017    4664     8714     3564     3150     6205     3072
##      treated3
## FBgn0000003      1
## FBgn0000008     70
## FBgn0000014      0
## FBgn0000015      0
## FBgn0000017    3334
## 12354 more rows ...
##
## $samples
##      group lib.size norm.factors
## untreated1 untreated 13972512      1
## untreated2 untreated 21911438      1
## untreated3 untreated  8358426      1
## untreated4 untreated  9841335      1
## treated1   treated 18670279      1
## treated2   treated  9571826      1
## treated3   treated 10343856      1
```

1.2. Normalizamos los datos utilizando el método TMM y hacemos la transformación logarítmica. NOTA: almacena el resultado de la normalización en el mismo objeto `DGEList` para que se actualicen los factores de normalización.

Solución:

```
dge <- calcNormFactors(dge, method = "TMM")
pseudoCount <- log2(cpm(dge) + 1)
```

1.3. Previamente a realizar el contraste es necesario estimar dos parámetros de dispersión del modelo: una varianza común y otra específica de cada gen. Estas estimaciones deben incorporarse al objeto `DGEList` ya creado. La función `estimateCommonDisp()` se utiliza para estimar la varianza común y `estimateTagwiseDisp()` para estimar la dispersión específica de cada gen. Ambas funciones requieren como argumento el objeto `DGEList`.

Solución:

```
dge <- estimateCommonDisp(dge)
dge <- estimateTagwiseDisp(dge)
dge
```

```
## An object of class "DGEList"
## $counts
##      untreated1 untreated2 untreated3 untreated4 treated1 treated2
## FBgn0000003      0      0      0      0      0      0
## FBgn0000008     92     161      76      70     140     88
## FBgn0000014      5      1      0      0      4      0
## FBgn0000015      0      2      1      2      1      0
## FBgn0000017    4664     8714     3564     3150    6205    3072
##      treated3
## FBgn0000003      1
## FBgn0000008     70
## FBgn0000014      0
## FBgn0000015      0
## FBgn0000017    3334
## 12354 more rows ...
##
## $samples
##      group lib.size norm.factors
## untreated1 untreated 13972512  0.9995731
## untreated2 untreated 21911438  1.0081519
## untreated3 untreated  8358426  0.9843974
## untreated4 untreated  9841335  0.9525077
## treated1   treated 18670279  1.0651817
## treated2   treated  9571826  0.9957012
## treated3   treated 10343856  0.9978557
##
## $common.dispersion
## [1] 0.0223401
##
## $pseudo.counts
##      untreated1 untreated2 untreated3 untreated4 treated1
## FBgn0000003 1.387779e-17 1.387779e-17 1.387779e-17 1.387779e-17  0.000000
## FBgn0000008 8.183634e+01 9.048388e+01 1.135300e+02 9.311235e+01  87.189060
## FBgn0000014 4.651902e+00 3.316442e-01 1.275938e-01 9.009287e-02  2.818797
## FBgn0000015 0.000000e+00 1.075299e+00 1.493017e+00 2.479509e+00  0.667060
## FBgn0000017 4.159717e+03 4.914570e+03 5.394127e+03 4.186859e+03 3886.556726
##      treated2 treated3
## FBgn0000003 1.957929e-02 1.142677e+00
## FBgn0000008 1.141053e+02 8.482551e+01
## FBgn0000014 7.599912e-02 5.435212e-02
## FBgn0000015 1.947578e-02 1.394236e-02
## FBgn0000017 4.015267e+03 4.023768e+03
## 12354 more rows ...
##
## $pseudo.lib.size
## [1] 12457236
##
## $AveLogCPM
## [1] -2.636799 2.953163 -1.966390 -2.223020 8.454622
## 12354 more elements ...
##
## $prior.df
## [1] 10
##
## $prior.n
## [1] 2
##
## $tagwise.dispersion
## [1] 0.08149572 0.02841098 0.63293562 0.07769978 0.01441984
## 12354 more elements ...
##
## $span
## [1] 0.3
```

1.4. Utilizamos la función `exactTest()` para encontrar los genes que están diferencialmente expresados. Esta función requiere como argumento un objeto de la clase `DGEList`.

Solución:

```
dgeExactTest <- exactTest(dge)
dgeExactTest
```

```
## An object of class "DGEEExact"
## $table
##          logFC  logCPM   PValue
## FBgn0000003 -1.89513707 -2.636799 0.85714286
## FBgn0000008 -0.01093402  2.953163 0.97881158
## FBgn0000014  0.33559344 -1.966390 1.00000000
## FBgn0000015  1.59744972 -2.223020 0.39984317
## FBgn0000017  0.23043641  8.454622 0.08656493
## 12354 more rows ...
##
## $comparison
## [1] "treated"  "untreated"
##
## $genes
## NULL
```

1.5. Para extraer la información de los genes diferencialmente expresados utilizamos la función `topTags()`. Esta función tiene los siguientes argumentos:

- objeto del tipo `DGEEExact` o `DGELRT` que contiene los estadísticos test y los p-valores.
- `n`, un entero que indica el número de genes a recuperar
- `adjust.method`, por defecto se utiliza el método de Benjamini & Hochberg, pero es posible utilizar otros métodos de ajuste (ver la ayuda de la función).
- `sort.by` que puede ser: “PValue” para ordenar los genes por su p-valor (valor por defecto), “logFC” para hacerlo por el valor absoluto del logFC o “none” para no ordenarlo.
- `p.value`, un umbral de corte para el p-valor ajustado

Ordena todos los genes por su p-valor. ¿Qué gen es el más significativo?

Solución:

```
resExactTest <- topTags(dgeExactTest, n = nrow(dgeExactTest$table))
head(resExactTest$table)
```

```
##          logFC  logCPM   PValue   FDR
## FBgn0039155  4.603245 5.882327 2.049796e-140 2.533342e-136
## FBgn0025111 -2.906160 6.923415 4.095017e-105 2.530516e-101
## FBgn0003360  3.173027 8.452788 3.507045e-79 1.444786e-75
## FBgn0039827  4.142741 4.397803 8.851535e-68 2.734903e-64
## FBgn0035085  2.549465 5.684878 2.875918e-62 7.108694e-59
## FBgn0029167  2.187893 8.221271 3.624446e-62 7.465755e-59
```

1.6. El objeto `TopTags` tiene un elemento, `table`, que es un `data.frame` con toda la información. Selecciona los genes diferencialmente expresados utilizando el criterio de: p-valor ajustado significativo a nivel 0.05 y un log fold change mayor que 1. ¿Cuántos genes están diferencialmente expresados? ¿Cuántos se sobre-expresan?

Solución:

```
selectedET <- resExactTest$table$FDR < 0.05 & abs(resExactTest$table$logFC) > 1
selectedET <- resExactTest$table[selectedET, ]
nrow(selectedET)
```

```
## [1] 267
```

```
head(selectedET)
```

```
##          logFC  logCPM   PValue   FDR
## FBgn0039155  4.603245 5.882327 2.049796e-140 2.533342e-136
## FBgn0025111 -2.906160 6.923415 4.095017e-105 2.530516e-101
## FBgn0003360  3.173027 8.452788 3.507045e-79 1.444786e-75
## FBgn0039827  4.142741 4.397803 8.851535e-68 2.734903e-64
## FBgn0035085  2.549465 5.684878 2.875918e-62 7.108694e-59
## FBgn0029167  2.187893 8.221271 3.624446e-62 7.465755e-59
```

Encontramos 267 genes diferencialmente expresados. Podemos utilizar la columna `logFC` para añadir la información sobre si estan sobre- o infra-expresados.

```
selectedET$updown <- factor(ifelse(selectedET$logFC > 0, "up", "down"))
head(selectedET)
```



```
##      logFC  logCPM      PValue      FDR updown
## FBgn0039155  4.603245 5.882327 2.049796e-140 2.533342e-136   up
## FBgn0025111 -2.906160 6.923415 4.095017e-105 2.530516e-101  down
## FBgn0003360  3.173027 8.452788 3.507045e-79 1.444786e-75   up
## FBgn0039827  4.142741 4.397803 8.851535e-68 2.734903e-64   up
## FBgn0035085  2.549465 5.684878 2.875918e-62 7.108694e-59   up
## FBgn0029167  2.187893 8.221271 3.624446e-62 7.465755e-59   up
```

```
table(selectedET$updown)
```

```
##
## down up
## 150 117
```

Modelo lineal generalizado con condición y réplica

Con esta aproximación ajustamos un modelo lineal generalizado. Este tipo de aproximaciones nos van a permitir tener en cuenta otros efectos como el efecto de la réplica.

El punto de partida es el objeto de tipo `DGEList`, creado a partir de la matriz de conteo original y en el que incorporamos la información sobre el tipo de muestra: `condition`. Utiliza el argumento `remove.zeros=TRUE` para eliminar los genes que no se han expresado en ninguna de las muestras.

```
condition<-as.factor(gsub("[0-9]", "", colnames(rawCountTable)))
dge <- DGEList(rawCountTable, group = condition, remove.zeros = TRUE)
dge <- calcNormFactors(dge, method = "TMM")
```

Añadimos la información de la réplica en el elemento `samples` como una nueva variable `replicate`,

```
dge$samples$replicate<-as.factor(c(1:4,1:3))
```

2.1. Define una matriz de diseño con la función `model.matrix()` incorporando dos factores: el grupo y la réplica, junto con la constante (intercept).

Solución:

```
design.matrix <- model.matrix(~ dge$samples$group +
                             dge$samples$replicate)
design.matrix
```

```
## (Intercept) dge$samples$groupuntreated dge$samples$replicate2
## 1      1      1      0
## 2      1      1      1
## 3      1      1      0
## 4      1      1      0
## 5      1      0      0
## 6      1      0      1
## 7      1      0      0
## dge$samples$replicate3 dge$samples$replicate4
## 1      0      0
## 2      0      0
## 3      1      0
## 4      0      1
## 5      0      0
## 6      0      0
## 7      1      0
## attr(,"assign")
## [1] 0 1 2 2 2
## attr(,"contrasts")
## attr(,"contrasts")$`dge$samples$group`
## [1] "contr.treatment"
##
## attr(,"contrasts")$`dge$samples$replicate`
## [1] "contr.treatment"
```

2.2. Estimamos los parámetros de dispersión. Utilizamos la función `estimateDisp()`, que tiene como argumentos el objeto `DGEList` y la matriz de diseño que acabamos de crear. Incorporamos la información al objeto `dge`.

Solución:

```
dge <- estimateDisp(dge, design.matrix)
dge
```

```
## An object of class "DGEList"
## $counts
##      untreated1 untreated2 untreated3 untreated4 treated1 treated2
## FBgn0000003      0      0      0      0      0      0
## FBgn0000008     92     161      76      70     140     88
## FBgn0000014      5      1      0      0      4      0
## FBgn0000015      0      2      1      2      1      0
## FBgn0000017    4664     8714     3564     3150     6205     3072
##      treated3
## FBgn0000003      1
## FBgn0000008     70
## FBgn0000014      0
## FBgn0000015      0
## FBgn0000017    3334
## 12354 more rows ...
##
## $samples
##      group lib.size norm.factors replicate
## untreated1 untreated 13972512  0.9995731      1
## untreated2 untreated 21911438  1.0081519      2
## untreated3 untreated  8358426  0.9843974      3
## untreated4 untreated  9841335  0.9525077      4
## treated1   treated 18670279  1.0651817      1
## treated2   treated  9571826  0.9957012      2
## treated3   treated 10343856  0.9978557      3
##
## $design
## (Intercept) dge$samples$groupuntreated dge$samples$replicate2
## 1      1      1      0
## 2      1      1      1
## 3      1      1      0
## 4      1      1      0
## 5      1      0      0
## 6      1      0      1
## 7      1      0      0
## dge$samples$replicate3 dge$samples$replicate4
## 1      0      0
## 2      0      0
## 3      1      0
## 4      0      1
## 5      0      0
## 6      0      0
## 7      1      0
## attr(,"assign")
## [1] 0 1 2 2 2
## attr(,"contrasts")
## attr(,"contrasts")$`dge$samples$group`
## [1] "contr.treatment"
##
## attr(,"contrasts")$`dge$samples$replicate`
## [1] "contr.treatment"
##
##
## $common.dispersion
## [1] 0.0210948
##
## $trended.dispersion
## [1] 0.13632190 0.03601234 0.13461607 0.13586830 0.01603072
## 12354 more elements ...
##
## $tagwise.dispersion
## [1] 0.13632190 0.03430280 0.12621556 0.14194010 0.01450993
## 12354 more elements ...
##
## $AveLogCPM
## [1] -2.636886  2.952682 -1.966067 -2.223044  8.454615
## 12354 more elements ...
##
## $trend.method
## [1] "lofit"
##
## $prior.df
## [1] 11.12077
##
```

```
## $prior.n
## [1] 5.560384
##
## $span
## [1] 0.3013916
```

2.3. Ajusta el modelo con la función `glmFit()` .

Solución:

```
fit <- glmFit(dge, design.matrix)
```

2.4. Hacemos el contraste utilizando el test de razón de verosimilitudes con la función `glmRT()` . Esta función requiere de dos argumentos, el objeto `fit` con los ajustes y un coeficiente que depende del contraste a realizar. En este caso, interesa contrastar la diferencia entre tratado y no tratado, lo que es equivalente a contrastar que el segundo coeficiente del modelo (`untreated` en la matriz de diseño) es 0 (argumento `coef=2`).

Solución:

```
dgeLRTtest <- glmLRT(fit, coef = 2)
dgeLRTtest
```

```
## An object of class "DGELRT"
## $coefficients
##      (Intercept) dge$samples$groupuntreated dge$samples$replicate2
## FBgn0000003    -18.14874          -1.14848953          0.31277005
## FBgn0000008    -11.88993          -0.00876597          0.18152239
## FBgn0000014    -15.41687          0.63747346         -2.08861127
## FBgn0000015    -17.41699          0.73674020          0.38746103
## FBgn0000017     -8.13040          0.18753227          0.09778968
##      dge$samples$replicate3 dge$samples$replicate4
## FBgn0000003         1.9215629         0.81131613
## FBgn0000008         0.1559792         0.09499937
## FBgn0000014        -3.4016208        -3.70651677
## FBgn0000015         0.4358833         1.36292154
## FBgn0000017         0.1469162        -0.05539184
## 12354 more rows ...
##
## $fitted.values
##      untreated1 untreated2 untreated3 untreated4  treated1
## FBgn0000003 3.347558e-15 6.575669e-15 8.617223e-08 3.771900e-09 4.409517e-08
## FBgn0000008 9.483114e+01 1.798820e+02 6.531037e+01 7.000000e+01 1.362237e+02
## FBgn0000014 5.284583e+00 8.117921e-01 1.229052e-07 9.108647e-08 3.748514e+00
## FBgn0000015 6.493117e-01 1.656954e+00 6.538096e-01 2.000000e+00 3.739269e-01
## FBgn0000017 4.960599e+03 8.651920e+03 3.384905e+03 3.150000e+03 5.855624e+03
##      treated2  treated3
## FBgn0000003 2.624486e-08 1.000000e+00
## FBgn0000008 7.829442e+01 8.265203e+01
## FBgn0000014 1.744756e-01 7.680480e-08
## FBgn0000015 2.891249e-01 3.317069e-01
## FBgn0000017 3.094519e+03 3.520103e+03
## 12354 more rows ...
##
## $deviance
## [1] 2.093789e-07 1.688757e+00 4.023025e-01 3.316410e+00 8.720006e-01
## 12354 more elements ...
##
## $iter
## [1] 17 4 16 6 4
## 12354 more elements ...
##
## $failed
## [1] FALSE FALSE FALSE FALSE FALSE
## 12354 more elements ...
##
## $method
## [1] "levenberg"
##
## $unshrunk.coefficients
##      (Intercept) dge$samples$groupuntreated dge$samples$replicate2
## FBgn0000003   -33.742504          -16.040216372          0.21668241
## FBgn0000008   -11.891291          -0.008787107          0.18174051
## FBgn0000014   -15.484229          0.696847487         -2.33176745
## FBgn0000015   -17.789284          0.905265873          0.47836097
## FBgn0000017    -9.129421          0.187532142          0.09778968
```

```

## FBgn0000017 -8.130431 0.187537149 0.09779223
## dge$samples$replicate3 dge$samples$replicate4
## FBgn0000003 17.5927477 14.3335889
## FBgn0000008 0.1561734 0.0951282
## FBgn0000014 -17.0475261 -17.4775196
## FBgn0000015 0.5360238 1.5237204
## FBgn0000017 0.1469199 -0.0553932
## 12354 more rows ...
##
## $df.residual
## [1] 2 2 2 2 2
## 12354 more elements ...
##
## $design
## (Intercept) dge$samples$groupuntreated dge$samples$replicate2
## 1 1 1 0
## 2 1 1 1
## 3 1 1 0
## 4 1 1 0
## 5 1 0 0
## 6 1 0 1
## 7 1 0 0
## dge$samples$replicate3 dge$samples$replicate4
## 1 0 0
## 2 0 0
## 3 1 0
## 4 0 1
## 5 0 0
## 6 0 0
## 7 1 0
## attr(,"assign")
## [1] 0 1 2 2 2
## attr(,"contrasts")
## attr(,"contrasts")$`dge$samples$group`
## [1] "contr.treatment"
##
## attr(,"contrasts")$`dge$samples$replicate`
## [1] "contr.treatment"
##
##
## $offset
## [,1] [,2] [,3] [,4] [,5] [,6] [,7]
## [1,] 16.45218 16.91064 15.92306 16.05344 16.80559 16.07003 16.14976
## attr(,"class")
## [1] "CompressedMatrix"
## attr(,"Dims")
## [1] 5 7
## attr(,"repeat.row")
## [1] TRUE
## attr(,"repeat.col")
## [1] FALSE
## 12354 more rows ...
##
## $dispersion
## [1] 0.13632190 0.03430280 0.12621556 0.14194010 0.01450993
## 12354 more elements ...
##
## $prior.count
## [1] 0.125
##
## $samples
## group lib.size norm.factors replicate
## untreated1 untreated 13972512 0.9995731 1
## untreated2 untreated 21911438 1.0081519 2
## untreated3 untreated 8358426 0.9843974 3
## untreated4 untreated 9841335 0.9525077 4
## treated1 treated 18670279 1.0651817 1
## treated2 treated 9571826 0.9957012 2
## treated3 treated 10343856 0.9978557 3
##
## $prior.df
## [1] 11.12077
##
## $AveLogCPM
## [1] -2.636886 2.952682 -1.966067 -2.223044 8.454615
## 12354 more elements ...
##
## $table
## log2FC log2CPM log2RPKM log2RPKM + 1

```

```
##          logFC  logCPM    LR    PValue
## FBgn0000003 -1.65692014 -2.636886 1.122112091 0.28946412
## FBgn0000008 -0.01264662  2.952682 0.002576786 0.95951513
## FBgn0000014  0.91967981 -1.966067 0.768642943 0.38063720
## FBgn0000015  1.06289144 -2.223044 0.588232897 0.44310326
## FBgn0000017  0.27055188  8.454615 3.566304577 0.05896339
## 12354 more rows ...
##
## $comparison
## [1] "dge$samples$groupuntreated"
##
## $df.test
## [1] 1 1 1 1 1
## 12354 more elements ...
```

2.5. Para obtener un listado de los genes diferencialmente expresados entre las dos condiciones (tratado y no tratado), utilizamos la función `topTags()` , seleccionando posteriormente los que verifiquen el criterio correspondiente. Selecciona los genes diferencialmente expresados utilizando el criterio de que su p-valor ajustado sea menor que 0.05 y el valor absoluto del logFC sea mayor que 1.

Solución:

```
resLRT <- topTags(dgeLRTtest, n = nrow(dge$counts))
head(resLRT$table)
```

```
##          logFC  logCPM    LR    PValue    FDR
## FBgn0039155  4.604087 5.882352 539.3326 2.633997e-119 3.255357e-115
## FBgn0003360  3.201303 8.452818 385.0931 9.685618e-86 5.985228e-82
## FBgn0025111 -2.908066 6.923382 350.8788 2.727564e-78 1.123666e-74
## FBgn0039827  4.128665 4.397399 308.4834 4.673034e-69 1.443851e-65
## FBgn0034736  3.483673 4.186495 241.2950 2.052728e-54 5.073932e-51
## FBgn0035085  2.524346 5.684768 230.6874 4.221147e-52 8.694859e-49
```

```
selectedLRT <- resLRT$table$FDR < 0.05 & abs(resLRT$table$logFC) > 1
selectedLRT <- resLRT$table[selectedLRT, ]
nrow(selectedLRT)
```

```
## [1] 246
```

```
head(selectedLRT)
```

```
##          logFC  logCPM    LR    PValue    FDR
## FBgn0039155  4.604087 5.882352 539.3326 2.633997e-119 3.255357e-115
## FBgn0003360  3.201303 8.452818 385.0931 9.685618e-86 5.985228e-82
## FBgn0025111 -2.908066 6.923382 350.8788 2.727564e-78 1.123666e-74
## FBgn0039827  4.128665 4.397399 308.4834 4.673034e-69 1.443851e-65
## FBgn0034736  3.483673 4.186495 241.2950 2.052728e-54 5.073932e-51
## FBgn0035085  2.524346 5.684768 230.6874 4.221147e-52 8.694859e-49
```

En este caso encontramos 246 genes diferencialmente expresados.

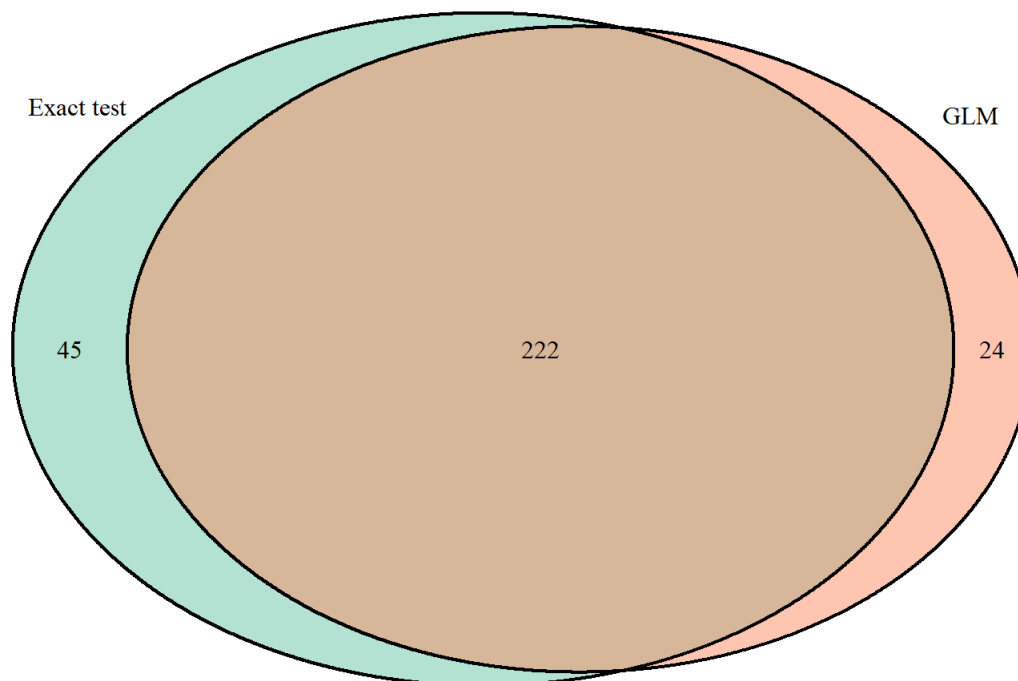
Comparación de los resultados

Podemos utilizar un diagrama de Venn para evaluar la coincidencia de los genes encontrados como diferencialmente expresados con los dos métodos. Utilizamos la función `venn.diagram()` disponible en el paquete `VennDiagram` de CRAN.

```
if(!("VennDiagram" %in% installed.packages()))
  install.packages("VennDiagram")

library("VennDiagram")

vd <- venn.diagram(x = list("Exact test" = rownames(selectedET),
                           "GLM" = rownames(selectedLRT)),
                  fill = RColorBrewer::brewer.pal(3, "Set2")[1:2], filename = NULL)
grid.draw(vd)
```

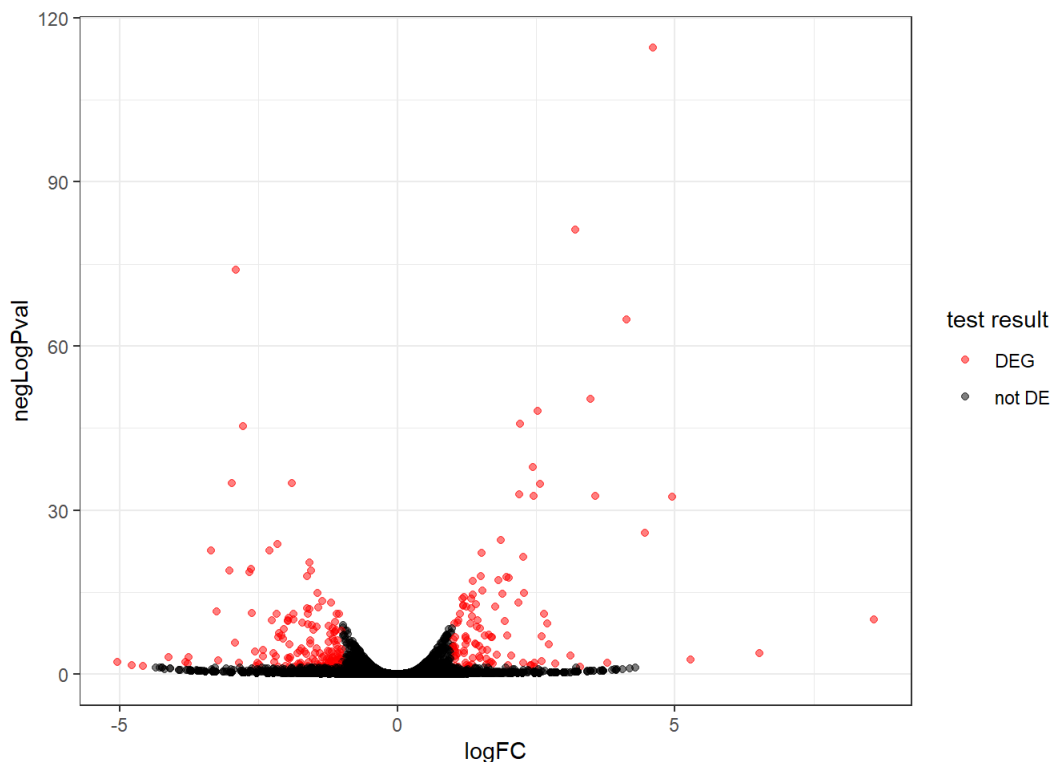


Vemos que 222 genes se identificarían como diferencialmente expresados con ambos métodos.

Exploración de los resultados del análisis de expresión diferencial

Volcano plot

```
volcanoData <- data.frame(logFC=resLRT$table$logFC, negLogPval=-log10(resLRT$table$FDR))
DEGs <- resLRT$table$FDR < 0.05 & abs(resLRT$table$logFC) > 1
volcanoData$selected<-ifelse(DEGs, "DEG", "not DE")
p <- ggplot(volcanoData, aes(x = logFC, y = negLogPval, colour = selected)) +
  geom_point(alpha = 0.5) + theme_bw() +
  scale_colour_manual(name = "test result", values = c("red", "black"))
p
```



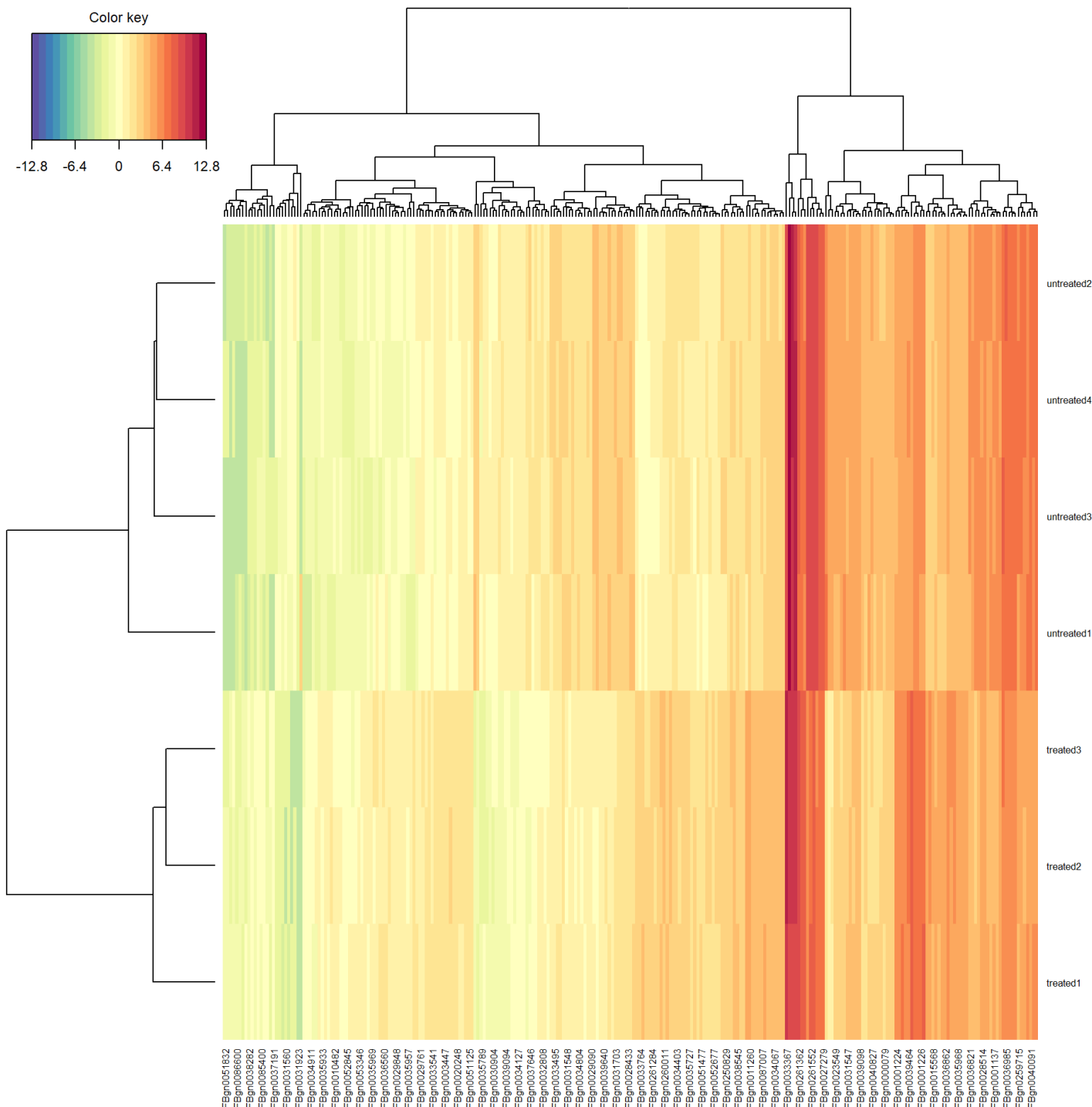
Clustering

Utilizamos la función `cim()` del paquete `mixOmics` disponible en Bioconductor.

```
if(!("mixOmics" %in% installed.packages()))
  BiocManager::install("mixOmics")
library("mixOmics")
```

Representamos los conteos transformados logarítmica de los genes encontrados como diferencialmente expresados. Utilizamos la función `cpm()` para transformar los conteos.

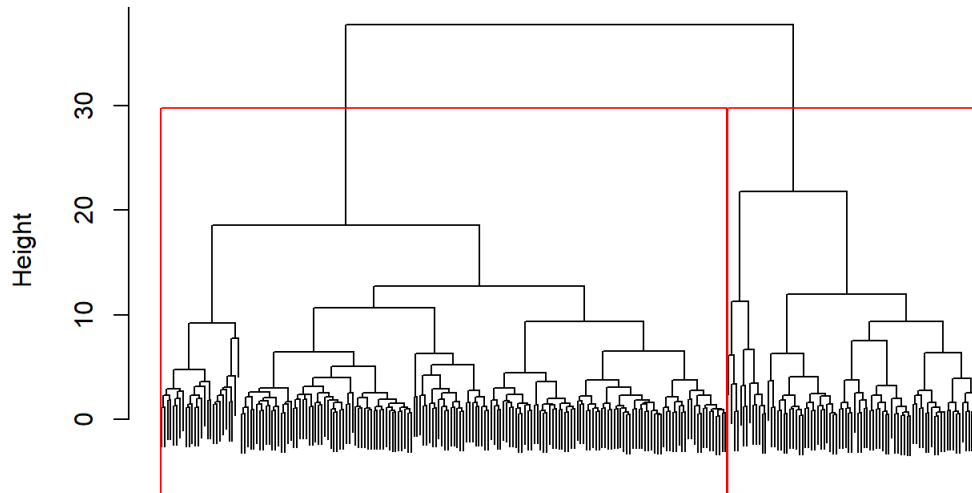
```
selY <- cpm(dge, log = TRUE, prior.count = 1)
# seleccionados los genes diferencialmente expresados
selY <- selY[match(rownames(selectedET), rownames(dge$counts)), ]
res.cim <- cim(t(selY), row.cex = 0.7, col.cex = 0.7)
```



Podemos representar el dendrograma en un gráfico independiente y marcar un número de clusters en este gráfico. Por ejemplo, para dos clusters,

```
res.hac <- as.hclust(res.cim$ddc)
plot(res.hac, labels = FALSE, xlab = "", sub = "", main = "Groups of genes")
rect.hclust(res.hac, k = 2, border = "red")
```

Groups of genes



Agrupamos los genes en los 2 clustes identificados utilizando la función `cutree()` .

```
genes.group <- cutree(res.hac, k = 2)
table(genes.group)
```

```
## genes.group
##  1  2
## 83 184
```

```
# genes del cluster 1
names(which(genes.group == 1))
```

```
## [1] "FBgn0039155" "FBgn0025111" "FBgn0003360" "FBgn0039827" "FBgn0035085"
## [6] "FBgn0029167" "FBgn0034736" "FBgn0029896" "FBgn0000071" "FBgn0034897"
## [11] "FBgn0003501" "FBgn0038832" "FBgn0040091" "FBgn0026562" "FBgn0023479"
## [16] "FBgn0052407" "FBgn0001226" "FBgn0040827" "FBgn0001224" "FBgn0034010"
## [21] "FBgn0261552" "FBgn0037754" "FBgn0027279" "FBgn0039419" "FBgn0040099"
## [26] "FBgn0063649" "FBgn0016715" "FBgn0261362" "FBgn0033913" "FBgn0001225"
## [31] "FBgn0003748" "FBgn0039109" "FBgn0038149" "FBgn0036985" "FBgn0034751"
## [36] "FBgn0031327" "FBgn0063498" "FBgn0031912" "FBgn0039257" "FBgn0000116"
## [41] "FBgn0031150" "FBgn0001137" "FBgn0033367" "FBgn0031322" "FBgn0032004"
## [46] "FBgn0035968" "FBgn0000406" "FBgn0032598" "FBgn0039464" "FBgn0029002"
## [51] "FBgn0053307" "FBgn0023549" "FBgn0259715" "FBgn0000567" "FBgn0035147"
## [56] "FBgn0040388" "FBgn0030805" "FBgn0034718" "FBgn0035266" "FBgn0052135"
## [61] "FBgn0033635" "FBgn0015568" "FBgn0038865" "FBgn0004396" "FBgn0031117"
## [66] "FBgn0037635" "FBgn0032638" "FBgn0028514" "FBgn0036821" "FBgn0031547"
## [71] "FBgn0259224" "FBgn0033782" "FBgn0043364" "FBgn0261015" "FBgn0039098"
## [76] "FBgn0038381" "FBgn0027348" "FBgn0036576" "FBgn0030326" "FBgn0036862"
## [81] "FBgn0259714" "FBgn0053556" "FBgn0000079"
```

Loading [MathJax]/jax/output/HTML-CSS/jax.js